

Build a Smarter Search with LangChain Context Retrieval

Estimated time needed: **60** minutes

Table of Contents

1. Overview
2. Objectives
3. Setup
    A. Installing required libraries
    B. Defining helper functions
4. Creating a retriever model
    A. Build the LLM
    B. Use the text splitter
    C. Create the embedding model
    D. Use Retrievers
        a. Vector Store-Backed Retriever
        b. Multi-Query Retriever
        c. Self-Querying Retriever
        d. Parent Document Retriever
5. Exercises
    A. Retrieve Top 2 Results Using Vector Store-Backed Retriever
    B. Self-Querying Retriever for a Query

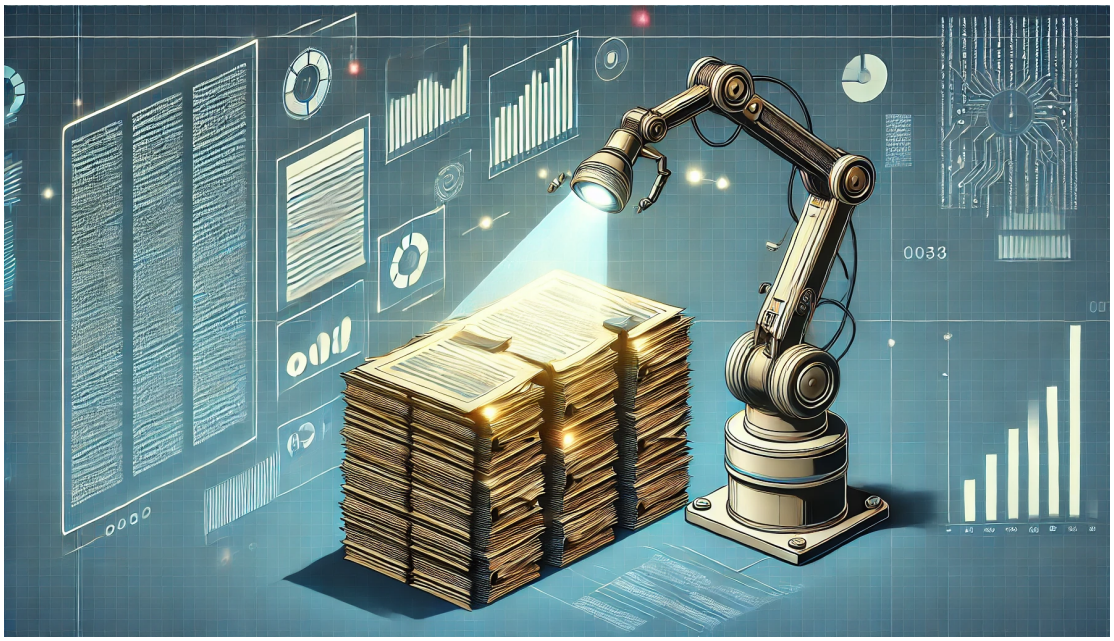
Overview

Imagine you are working on a project that involves processing a large collection of text documents, such as research papers, legal documents, or customer service logs. Your task is to develop a system that can quickly retrieve the most relevant segments of text based on a user's query. Traditional keyword-based search methods might not be sufficient, as they often fail to capture the nuanced meanings and contexts within the documents. To address this challenge, you can use different types of retrievers based on LangChain.

Using retrievers is crucial for several reasons:

- **Efficiency:** Retrievers enable fast and efficient retrieval of relevant information from large datasets, saving time and computational resources.

- **Accuracy:** By leveraging advanced retrieval techniques, these tools can provide more accurate and contextually relevant results compared to traditional search methods.
- **Versatility:** Different retrievers can be tailored to specific use cases, making them adaptable to various types of text data and query requirements.
- **Context awareness:** Some retrievers, such as the Parent Document Retriever, can consider the broader context of the document, enhancing the relevance of the retrieved segments.



In this lab, you will learn how to use various retrievers to efficiently extract relevant document segments from text using LangChain. You will learn about four types of retrievers: `Vector Store-backed Retriever`, `Multi-Query Retriever`, `Self-Querying Retriever`, and `Parent Document Retriever`. You will also learn the differences between these retrievers and understand the appropriate situations in which to use each one. By the end of this lab, you will be equipped with the skills to implement and utilize these retrievers in your projects.

Objectives

After completing this lab, you will be able to:

- Use various types of retrievers to efficiently extract relevant document segments from text, leveraging LangChain's capabilities.
- Apply the Vector Store-backed Retriever to solve problems involving semantic similarity and relevance in large text datasets.
- Utilize the Multi-Query Retriever to address situations where multiple query variations are needed to capture comprehensive results.
- Implement the Self-Querying Retriever to automatically generate and refine queries, enhancing the accuracy of information retrieval.
- Employ the Parent Document Retriever to maintain context and relevance by

considering the broader context of the parent document.

Setup

For this lab, you will use the following libraries:

- `ibm-watson-ai` for using LLMs from IBM's watsonx.ai.
- `langchain`, `langchain-ibm`, `langchain-community` for using relevant features from LangChain.
- `pypdf` is an open-source pure Python PDF library capable of splitting, merging, cropping, and transforming the pages of PDF files.
- `chromadb` is an open-source vector database used to store embeddings.
- `lark` is a general-purpose parsing library for Python. It is necessary for a Self-Querying Retriever.

Installing required libraries

The following required libraries are **not** preinstalled in the Skills Network Labs environment. **You must run the following cell** to install them:

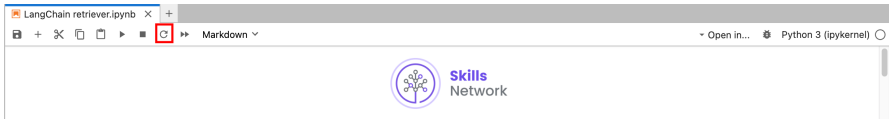
Note: The version is being pinned here to specify the version. It's recommended that you do this as well. Even if the library is updated in the future, the installed library could still support this lab work.

This might take approximately 1-2 minutes.

In [1]:
```
!pip install "ibm-watsonx-ai==1.1.2" | tail -n 1
!pip install "langchain==0.2.1" | tail -n 1
!pip install "langchain-ibm==0.1.11" | tail -n 1
!pip install "langchain-community==0.2.1" | tail -n 1
!pip install "chromadb==0.4.24" | tail -n 1
!pip install "pypdf==4.3.1" | tail -n 1
!pip install "lark==1.1.9" | tail -n 1
!pip install 'posthog<6.0.0' | tail -n 1
```

```
Successfully installed ibm-cos-sdk-2.13.6 ibm-cos-sdk-core-2.13.6 ibm-cos-
sdk-s3transfer-2.13.6 ibm-watsonx-ai-1.1.2 jmespath-1.0.1 lomond-0.3.3 num
py-1.26.4 pandas-2.1.4 requests-2.32.2 tabulate-0.10.0 tzdata-2026.3
Successfully installed langchain-0.2.1 langchain-core-0.2.43 langchain-tex
t-splitters-0.2.4 langsmith-0.1.147 orjson-3.11.9 requests-toolbelt-1.0.0
tenacity-8.5.0
Successfully installed langchain-ibm-0.1.11
Successfully installed dataclasses-json-0.6.7 langchain-community-0.2.1 ma
rshmallow-3.26.2 mypy-extensions-1.1.0 typing-inspect-0.9.0
Successfully installed PyYAML-6.0.3 aiohttp-3.14.3 aiosignal-1.4.0 annotat
ed-doc-0.0.4 asgiref-3.12.1 backoff-2.2.1 bcrypt-5.0.0 build-1.5.0 chroma-
hnswlib-0.7.3 chromadb-0.4.24 click-8.4.2 durationpy-0.10 fastapi-0.140.4
filelock-3.32.0 flatbuffers-25.12.19 fsspec-2026.6.0 googleapis-common-pro
tos-1.75.0 grpcio-1.83.0 hf-xet-1.5.2 httptools-0.8.0 huggingface-hub-1.2
5.1 kubernetes-36.0.3 markdown-it-py-4.2.0 mdurl-0.1.2 mmh3-5.2.1 onnxrunt
ime-1.28.0 opentelemetry-api-1.44.0 opentelemetry-exporter-otlp-proto-comm
on-1.44.0 opentelemetry-exporter-otlp-proto-grpc-1.44.0 opentelemetry-inst
rumentation-0.65b0 opentelemetry-instrumentation-asgi-0.65b0 opentelemetr
y-instrumentation-fastapi-0.65b0 opentelemetry-proto-1.44.0 opentelemetry-
sdk-1.44.0 opentelemetry-semantic-conventions-0.65b0 opentelemetry-util-ht
tp-0.65b0 posthog-7.30.1 protobuf-7.35.1 pulsar-client-3.13.0 pypika-0.5
1.1 pyproject_hooks-1.2.0 python-dotenv-1.2.2 requests-oauthlib-2.0.0 ric
h-15.0.0 shellingham-1.5.4 starlette-1.3.1 tokenizers-0.23.1 typer-0.27.0
typing-inspection-0.4.2 uvicorn-0.51.0 uvloop-0.22.1 watchfiles-1.2.0 webs
ockets-16.1.1 wrapt-2.2.2
Successfully installed pypdf-4.3.1
Successfully installed lark-1.1.9
Successfully installed posthog-5.4.0
```

After you install the libraries, restart your kernel. You can do that by clicking the **Restart the kernel** icon.



Defining helper functions

Use the following code to define some helper functions to reduce the repeat work in the notebook:

In [1]:
```
# You can use this section to suppress warnings generated by your code:
def warn(*args, **kwargs):
    pass
import warnings
warnings.warn = warn
warnings.filterwarnings('ignore')
```

Creating a retriever model

The following steps are involved to create a retriever model using LangChain:

- Building LLMs
- Splitting documents into chunks

- Building an embedding model
- Retrieving related knowledge from text

Build the LLM

Develop or select a pre-trained language model that can understand and generate human-like text. This model serves as the foundation for processing and interpreting language data.

In [2]:
```
from ibm_watsonx_ai.foundation_models import ModelInference
from ibm_watsonx_ai.metanames import GenTextParamsMetaNames as GenParams
from ibm_watsonx_ai import Credentials
from ibm_watsonx_ai.foundation_models.extensions.langchain import Watsonx
```

The following will allow us to connect to watsonx.ai and set LLM parameters

In [3]:
```
def llm():
    model_id = 'mistralai/mistral-small-3-1-24b-instruct-2503'

    parameters = {
        GenParams.MAX_NEW_TOKENS: 256,  # this controls the maximum numbe
        GenParams.TEMPERATURE: 0.5, # this randomness or creativity of th
    }

    credentials = {
        "url": "https://us-south.ml.cloud.ibm.com"
    }


    project_id = "skills-network"

    model = ModelInference(
        model_id=model_id,
        params=parameters,
        credentials=credentials,
        project_id=project_id
    )

    mixtral_llm = WatsonxLLM(model = model)
    return mixtral_llm
```

Use the text splitter

Break down large documents into smaller, manageable pieces or chunks. This helps in processing and analyzing the text more efficiently, allowing the model to focus on specific sections rather than being overwhelmed by the entire document.

In [13]:
```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

In [14]:
```
def text_splitter(data, chunk_size, chunk_overlap):
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
```

```
        length_function=len,
    )
    chunks = text_splitter.split_documents(data)
    return chunks
```

Create the embedding model

Create or utilize an embedding model to convert chunks of text into numerical vectors. These vectors represent the semantic meaning of the text, enabling the model to compare and retrieve relevant information based on similarity. The following code demonstrates how to build an embedding model using the `watsonx.ai` package.

For this project, the `ibm/slate-125m-english-rtrvr-v2` embedding model is used.

In [15]:
```
from ibm_watsonx_ai.metanames import EmbedTextParamsMetaNames
from langchain_ibm import WatsonxEmbeddings
```

In [24]:
```
def watsonx_embedding():
    embed_params = {
        EmbedTextParamsMetaNames.TRUNCATE_INPUT_TOKENS: 3,
        EmbedTextParamsMetaNames.RETURN_OPTIONS: {"input_text": True},
    }

    watsonx_embedding = WatsonxEmbeddings(
        model_id="ibm/granite-embedding-278m-multilingual",
        url="https://us-south.ml.cloud.ibm.com",
        project_id="skills-network",
        params=embed_params,
    )
    return watsonx_embedding
```

Use Retrievers

A retriever is an interface designed to return documents based on an unstructured query. Unlike a vector store, which stores and retrieves documents, a retriever's primary function is to find and return relevant documents. While vector stores can serve as the backbone of a retriever, there are various other types of retrievers that can be used as well.

Retrievers take a string `query` as input and output a list of `Documents`.

Vector Store-Backed Retriever

A vector store retriever is a type of retriever that utilizes a vector store to fetch documents. It acts as a lightweight wrapper around the vector store class, enabling it to conform to the retriever interface. This retriever leverages the search methods implemented by the vector store, such as similarity search and Maximum Marginal Relevance (MMR), to query texts stored within it.

Before demonstrating this retriever, you need to load some example text. A `.txt`

document has been prepared for you.

In [7]:
```
!wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud
```

```
--2026-07-27 17:05:18--  https://cf-courses-data.s3.us.cloud-object-storag
e.appdomain.cloud/MZ9z1lm-Ui3YBp3SYWLTAQ/companypolicies.txt
Resolving cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-c
ourses-data.s3.us.cloud-object-storage.appdomain.cloud)... 198.23.119.245
connected. to cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud
(cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud)|198.23.119.24
5|:443...
HTTP request sent, awaiting response... 200 OK
Length: 15660 (15K) [text/plain]
Saving to: ‘companypolicies.txt’

companypolicies.txt 100%[===================>]  15.29K  --.-KB/s    in 0.0
3s

2026-07-27 17:05:19 (507 KB/s) - ‘companypolicies.txt’ saved [15660/15660]
```

Use `TextLoader` to load the document.

In [8]:
```
from langchain_community.document_loaders import TextLoader
```

In [9]:
```
loader = TextLoader("companypolicies.txt")
txt_data = loader.load()
```

Let's take a look at this document. This is a document about different policies in a company.

In [11]:
```
txt_data
```

Out[11]: [Document(metadata={'source': 'companypolicies.txt'}, page_content="1.\t Code of Conduct\n\nOur Code of Conduct outlines the fundamental principl es and ethical standards that guide every member of our organization. We are committed to maintaining a workplace that is built on integrity, res pect, and accountability.\nIntegrity: We hold ourselves to the highest e thical standards. This means acting honestly and transparently in all ou r interactions, whether with colleagues, clients, or the broader communi ty. We respect and protect sensitive information, and we avoid conflicts of interest.\nRespect: We embrace diversity and value each individual's contributions. Discrimination, harassment, or any form of disrespectful behavior is unacceptable. We create an inclusive environment where diffe rences are celebrated and everyone is treated with dignity and courtesy. \nAccountability: We take responsibility for our actions and decisions. We follow all relevant laws and regulations, and we strive to continuous ly improve our practices. We report any potential violations of this cod e and support the investigation of such matters.\nSafety: We prioritize the safety of our employees, clients, and the communities we serve. We m aintain a culture of safety, including reporting any unsafe conditions o r practices.\nEnvironmental Responsibility: We are committed to minimizi ng our environmental footprint and promoting sustainable practices.\nOur Code of Conduct is not just a set of rules; it is the foundation of our organization's culture. We expect all employees to uphold these principl es and serve as role models for others, ensuring we maintain our reputat ion for ethical conduct, integrity, and social responsibility.\n\n2.\tRe cruitment Policy\n\nOur Recruitment Policy reflects our commitment to at tracting, selecting, and onboarding the most qualified and diverse candi dates to join our organization. We believe that the success of our compa ny relies on the talents, skills, and dedication of our employees.\nEqua l Opportunity: We are an equal opportunity employer and do not discrimin ate on the basis of race, color, religion, sex, sexual orientation, gend er identity, national origin, age, disability, or any other protected st atus. We actively promote diversity and inclusion.\nTransparency: We mai ntain transparency in our recruitment processes. All job vacancies are a dvertised internally and externally when appropriate. Job descriptions a nd requirements are clear and accurately represent the role.\nSelection Criteria: Our selection process is based on the qualifications, experien ce, and skills necessary for the position. Interviews and assessments ar e conducted objectively, and decisions are made without bias.\nData Priv acy: We are committed to protecting the privacy of candidates' personal information and adhere to all relevant data protection laws and regulati ons.\nFeedback: Candidates will receive timely and constructive feedback on their application and interview performance.\nOnboarding: New employe es receive comprehensive onboarding to help them integrate into the orga nization effectively. This includes information on our culture, policies , and expectations.\nEmployee Referrals: We encourage and appreciate emp loyee referrals as they contribute to building a strong and engaged tea m.\nOur Recruitment Policy is a foundation for creating a diverse, inclu sive, and talented workforce. It ensures that we attract and hire the be st candidates who align with our company values and contribute to our co ntinued success. We continuously review and update this policy to reflec t evolving best practices in recruitment.\n\n3.\tInternet and Email Poli cy\n\nOur Internet and Email Policy is established to guide the responsi ble and secure use of these essential tools within our organization. We recognize their significance in daily business operations and the import ance of adhering to principles that maintain security, productivity, and legal compliance.\nAcceptable Use: Company-provided internet and email s ervices are primarily meant for job-related tasks. Limited personal use is allowed during non-work hours, provided it doesn't interfere with wor k responsibilities.\nSecurity: Safeguard your login credentials, avoidin g the sharing of passwords. Exercise caution with email attachments and

links from unknown sources. Promptly report any unusual online activity or potential security breaches.\nConfidentiality: Reserve email for the transmission of confidential information, trade secrets, and sensitive customer data only when encryption is applied. Exercise discretion when discussing company matters on public forums or social media.\nHarassment and Inappropriate Content: Internet and email usage must not involve harassment, discrimination, or the distribution of offensive or inappropriate content. Show respect and sensitivity to others in all online communications.\nCompliance: Ensure compliance with all relevant laws and regulations regarding internet and email usage, including those related to copyright and data protection.\nMonitoring: The company retains the right to monitor internet and email usage for security and compliance purposes.\nConsequences: Policy violations may lead to disciplinary measures, including potential termination.\nOur Internet and Email Policy aims to promote safe, responsible usage of digital communication tools that align with our values and legal obligations. Each employee is expected to understand and follow this policy. Regular reviews ensure its alignment with evolving technology and security standards.\n\n4.\tMobile Phone Policy\n\nThe Mobile Phone Policy sets forth the standards and expectations governing the appropriate and responsible usage of mobile devices in the organization. The purpose of this policy is to ensure that employees utilize mobile phones in a manner consistent with company values and legal compliance.\nAcceptable Use: Mobile devices are primarily intended for work-related tasks. Limited personal usage is allowed, provided it does not disrupt work obligations.\nSecurity: Safeguard your mobile device and access credentials. Exercise caution when downloading apps or clicking links from unfamiliar sources. Promptly report security concerns or suspicious activities related to your mobile device.\nConfidentiality: Avoid transmitting sensitive company information via unsecured messaging apps or emails. Be discreet when discussing company matters in public spaces.\nCost Management: Keep personal phone usage separate from company accounts and reimburse the company for any personal charges on company-issued phones.\nCompliance: Adhere to all pertinent laws and regulations concerning mobile phone usage, including those related to data protection and privacy.\nLost or Stolen Devices: Immediately report any lost or stolen mobile devices to the IT department or your supervisor.\nConsequences: Non-compliance with this policy may lead to disciplinary actions, including the potential loss of mobile phone privileges.\nThe Mobile Phone Policy is aimed at promoting the responsible and secure use of mobile devices in line with legal and ethical standards. Every employee is expected to comprehend and abide by these guidelines. Regular reviews of the policy ensure its ongoing alignment with evolving technology and security best practices.\n\n5.\tSmoking Policy\n\nPolicy Purpose: The Smoking Policy has been established to provide clear guidance and expectations concerning smoking on company premises. This policy is in place to ensure a safe and healthy environment for all employees, visitors, and the general public.\nDesignated Smoking Areas: Smoking is only permitted in designated smoking areas, as marked by appropriate signage. These areas have been chosen to minimize exposure to secondhand smoke and to maintain the overall cleanliness of the premises.\nSmoking Restrictions: Smoking inside company buildings, offices, meeting rooms, and other enclosed spaces is strictly prohibited. This includes electronic cigarettes and vaping devices.\nCompliance with Applicable Laws: All employees and visitors must adhere to relevant federal, state, and local smoking laws and regulations.\nDisposal of Smoking Materials: Properly dispose of cigarette butts and related materials in designated receptacles. Littering on company premises is prohibited.\nNo Smoking in Company Vehicles: Smoking is not permitted in company vehicles, whether they are owned or leased, to maintain the condition and cleanliness of these vehicles.\nEnforcement and Consequences: All employees and visitors are expected to adhere to this policy. No

n-compliance may lead to appropriate disciplinary action, which could in
clude fines, or, in the case of employees, possible termination of emplo
yment.\nReview of Policy: This policy will be reviewed periodically to e
nsure its alignment with evolving legal requirements and best practices
for maintaining a healthy and safe workplace.\nWe appreciate your cooper
ation in maintaining a smoke-free and safe environment for all.\n\n6.\tD
rug and Alcohol Policy\n\nPolicy Objective: The Drug and Alcohol Policy
is established to establish clear expectations and guidelines for the re
sponsible use of drugs and alcohol within the organization. This policy
aims to maintain a safe, healthy, and productive workplace.\nProhibited
Substances: The use, possession, distribution, or sale of illegal drugs
or unauthorized controlled substances is strictly prohibited on company
premises or during work-related activities. This includes the misuse of
prescription drugs.\nAlcohol Consumption: The consumption of alcoholic b
everages is not allowed during work hours, on company property, or while
performing company-related duties. Exception may be made for company-san
ctioned events.\nImpairment: Employees are expected to perform their job
duties without impairment from drugs or alcohol. The use of substances t
hat could impair job performance or pose a safety risk is prohibited.\nT
esting and Searches: The organization reserves the right to conduct drug
and alcohol testing as per applicable laws and regulations. Employees ma
y be subject to testing in cases of reasonable suspicion, post-accident,
or as part of routine workplace safety measures.\nReporting: Employees s
hould report any concerns related to drug or alcohol misuse by themselve
s or their colleagues, as well as safety concerns arising from such misu
se.\nTreatment and Assistance: Employees with substance abuse issues are
encouraged to seek help. The organization is committed to providing supp
ort, resources, and information to assist those seeking treatment.\nCons
equences: Violation of this policy may result in disciplinary actions, u
p to and including termination of employment. Legal action may also be p
ursued when necessary.\nPolicy Review: This policy will undergo periodic
review to ensure its continued relevance and compliance with evolving le
gal requirements and best practices for a safe and productive work envir
onment.\nYour adherence to this policy is appreciated as it helps to mai
ntain a safe and drug-free workplace for all.\n\n7.\tHealth and Safety P
olicy\n\nOur commitment to health and safety is paramount. We prioritize
the well-being of our employees, customers, and the public. We diligentl
y comply with all relevant health and safety laws and regulations. Our o
bjective is to maintain a workplace free from hazards, preventing accide
nts, injuries, and illnesses. Every individual within our organization i
s responsible for upholding these standards. We regularly assess and imp
rove our safety measures, provide adequate training, and encourage open
communication regarding safety concerns. Through collective dedication,
we aim to ensure a safe, healthy, and secure environment for all. Your c
ooperation is essential in achieving this common goal.\n\n8.\tAnti-discr
imination and Harassment Policy\n\nThe Anti-Discrimination and Harassmen
t Policy is a testament to the commitment of this organization in foster
ing a workplace that is free from discrimination, harassment, and any fo
rm of unlawful bias. This policy applies to every individual within the
organization, including employees, contractors, visitors, and clients.\n
Non-Discrimination: This organization strictly prohibits discrimination
based on race, color, religion, gender, national origin, age, disability
, sexual orientation, or any other legally protected characteristic in a
ll aspects of employment, including recruitment, hiring, compensation, b
enefits, promotions, and terminations.\nHarassment: Harassment in any fo
rm, whether based on the aforementioned characteristics or any other pro
tected status, is unacceptable. This encompasses unwelcome advances, off
ensive jokes, slurs, and other verbal or physical conduct that creates a
hostile or intimidating work environment.\nReporting: Individuals who ex
perience or witness any form of discrimination or harassment are encoura

```
ged to promptly report the incident to their supervisor, manager, or the
designated HR representative. The organization is committed to a timely
and confidential investigation of such complaints.\nConsequences: Violat
ion of this policy may result in disciplinary action, including terminat
ion of employment. The organization is committed to taking appropriate a
ction against any individual found to be in violation of this policy.\nR
eview and Update: This policy is subject to regular review and update to
remain aligned with evolving legal requirements and best practices in pr
eventing discrimination and harassment. This organization considers it a
collective responsibility to ensure a workplace free from discrimination
and harassment, and it is essential that every individual within the org
anization plays their part in upholding these principles.\n\n9.\tDiscipl
ine and Termination Policy\n\nThe Discipline and Termination Policy unde
rscores the organization's commitment to maintaining a productive, ethic
al, and respectful work environment. This policy applies to all personne
l, including employees, contractors, and temporary staff.\nPerformance a
nd Conduct Expectations: Employees are expected to meet performance stan
dards and adhere to conduct guidelines. The organization will provide cl
ear expectations, feedback, and opportunities for improvement when perfo
rmance or conduct issues arise.\nDisciplinary Actions: When necessary, d
isciplinary actions will be taken, which may include verbal warnings, wr
itten warnings, suspension, or other appropriate measures. Disciplinary
actions are designed to address issues constructively and maintain perfo
rmance standards.\nTermination: In situations where an employee's perfor
mance or conduct issues persist, the organization may resort to terminat
ion. Termination may also occur for reasons such as redundancy, violatio
n of policies, or restructuring.\nTermination Procedure: The organizatio
n will follow appropriate procedures, ensuring fairness and adherence to
legal requirements during the termination process. Employees may be elig
ible for notice periods, severance pay, or other benefits as per employm
ent agreements and applicable laws.\nExit Process: The organization will
conduct an exit process to ensure a smooth transition for departing empl
oyees, including the return of company property, final pay, and cancella
tion of access and benefits.\nThis policy serves as a framework for hand
ling discipline and termination. The organization recognizes the importa
nce of fairness and consistency in these processes, and decisions will b
e made after careful consideration. Every employee is expected to unders
tand and adhere to this policy, contributing to a respectful and product
ive workplace. Regular reviews will ensure its alignment with evolving l
egal requirements and best practices.\n")]
```

Split `txt_data` into chunks. `chunk_size = 200` , `chunk_overlap = 20` has been set.

In [17]:
```
chunks_txt = text_splitter(txt_data, 200, 20)
```

Store the embeddings into a `ChromaDB` .

In [18]:
```
from langchain.vectorstores import Chroma
```

In [25]:
```
vectordb = Chroma.from_documents(chunks_txt, watsonx_embedding())
```

Simple similarity search

Here is an example of a simple similarity search based on the vector database.

For this demonstration, the query has been set to "email policy".

In [26]:
```
query = "email policy"
retriever = vectordb.as_retriever()
```

In [27]:
```
docs = retriever.invoke(query)
```

By default, the number of retrieval results is four, and they are ranked by similarity level.

In [28]:
```
docs
```

Out[28]:
```
[Document(metadata={'source': 'companypolicies.txt'}, page_content='Empl
oyee Referrals: We encourage and appreciate employee referrals as they c
ontribute to building a strong and engaged team.'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='Acco
untability: We take responsibility for our actions and decisions. We fol
low all relevant laws and regulations, and we strive to continuously imp
rove our practices. We report any potential'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='Equa
l Opportunity: We are an equal opportunity employer and do not discrimin
ate on the basis of race, color, religion, sex, sexual orientation, gend
er identity, national origin, age, disability, or'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='form
of unlawful bias. This policy applies to every individual within the org
anization, including employees, contractors, visitors, and clients.')]
```

You can also specify `search kwargs` like `k` to limit the retrieval results.

In [29]:
```
retriever = vectordb.as_retriever(search_kwargs={"k": 1})
docs = retriever.invoke(query)
docs
```

Out[29]:
```
[Document(metadata={'source': 'companypolicies.txt'}, page_content='Empl
oyee Referrals: We encourage and appreciate employee referrals as they c
ontribute to building a strong and engaged team.')]
```

MMR search

MMR in vector stores is a technique used to balance the relevance and diversity of retrieved results. It selects documents that are both highly relevant to the query and minimally similar to previously selected documents. This approach helps to avoid redundancy and ensures a more comprehensive coverage of different aspects of the query.

The following code is showing how to conduct an MMR search in a vector database. You just need to sepecify `search_type="mmr"` .

In [31]:
```
retriever = vectordb.as_retriever(search_type="mmr")
docs = retriever.invoke(query)
docs
```

Out[31]:
```
[Document(metadata={'source': 'companypolicies.txt'}, page_content='Empl
oyee Referrals: We encourage and appreciate employee referrals as they c
ontribute to building a strong and engaged team.'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='Acco
untability: We take responsibility for our actions and decisions. We fol
low all relevant laws and regulations, and we strive to continuously imp
rove our practices. We report any potential'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='form
of unlawful bias. This policy applies to every individual within the org
anization, including employees, contractors, visitors, and clients.'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='onli
ne activity or potential security breaches.')]
```

Similarity score threshold retrieval

You can also set a retrieval method that defines a similarity score threshold, returning only documents with a score above that threshold.

In [33]:
```
retriever = vectordb.as_retriever(
    search_type="similarity_score_threshold", search_kwargs={"score_thres
)
docs = retriever.invoke(query)
docs
```

Out[33]:
```
[Document(metadata={'source': 'companypolicies.txt'}, page_content='Empl
oyee Referrals: We encourage and appreciate employee referrals as they c
ontribute to building a strong and engaged team.'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='Acco
untability: We take responsibility for our actions and decisions. We fol
low all relevant laws and regulations, and we strive to continuously imp
rove our practices. We report any potential'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='Equa
l Opportunity: We are an equal opportunity employer and do not discrimin
ate on the basis of race, color, religion, sex, sexual orientation, gend
er identity, national origin, age, disability, or'),
 Document(metadata={'source': 'companypolicies.txt'}, page_content='form
of unlawful bias. This policy applies to every individual within the org
anization, including employees, contractors, visitors, and clients.')]
```

Multi-Query Retriever

Distance-based vector database retrieval represents queries in high-dimensional space and finds similar embedded documents based on "distance". However, retrieval results may vary with subtle changes in query wording or if the embeddings do not accurately capture the data's semantics.

The `MultiQueryRetriever` addresses this by using an LLM to generate multiple queries from different perspectives for a given user input query. For each query, it retrieves a set of relevant documents and then takes the unique union of these results to form a larger set of potentially relevant documents. By generating multiple perspectives on the same question, the `MultiQueryRetriever` can potentially overcome some limitations of distance-based retrieval, resulting in a richer and more diverse set of results.

The following picture shows the difference between retrievers solely based on distance

and the Multi-Query Retriever.



Let's consider the query sentence, `"I like cats"` .

On the upper side of the picture, you can see a retriever that relies solely on distance. This retriever calculates the distance between the query and the documents in the vector store, returning the document with the closest match.

On the lower side, you can see a multi-query retriever. It first uses an LLM to generate multiple queries from different perspectives based on the user's input query. For each generated query, it retrieves relevant documents and then returns the union of these results.

A PDF document has been prepared to demonstrate this Multi-Query Retriever.

In [34]:
```
from langchain_community.document_loaders import PyPDFLoader
```

In [36]:
```
loader = PyPDFLoader("https://cf-courses-data.s3.us.cloud-object-storage.
pdf_data = loader.load()
```

Let's take a look at the first page of this paper. This paper is talking about the LangChain framework.

In [37]:
```
pdf_data[1]
```

Out[37]: Document(metadata={'source': 'https://cf-courses-data.s3.us.cloud-objec
t-storage.appdomain.cloud/ioch1wsxkfqgfLLgmd-6Rw/langchain-paper.pdf', '
page': 1}, page_content='LangChain helps us to unlock the ability to har
ness the \nLLM’s immense potential in tasks such as document analysis,
\nchatbot development, code analysis, and countless other \napplication
s. Whether your desire is to unlock deeper natural \nlanguage understand
ing , enhance data, or circumvent \nlanguage barriers through translatio
n, LangChain is ready to \nprovide the tools and programming support you
need to do \nwithout it that it is not only difficult but also fresh for
you . Its \ncore functionalities encompass: \n1. Context -Aware Capabil
ities: LangChain facilitates the \ndevelopment of applications that are
inherently \ncontext -aware. This means that these applications can \nco
nnect to a language model and draw from various \nsources of context, su
ch as prompt instructions, a few-\nshot examples, or existing content,
to ground their \nresponses effectively. \n2. Reasoning Abilities: Lang
Chain equips applications \nwith the capacity to reason effectively. By
relying on a \nlanguage model, these applications can make informed \nde
cisions about how to respond based on the provided \ncontext and determi
ne the appropriate acti ons to take. \nLangChain offers several key val
ue propositions: \nModular Components: It provides abstractions that \n
simplify working with language models, along with a \ncomprehensive coll
ection of implementations for each \nabstraction. These components are d
esigned to be modular \nand user -friendly, making them useful whethe r
you are \nutilizing the entire LangChain framework or not. \nOff-the-Sh
elf Chains: LangChain offers pre -configured \nchains, which are structu
red assemblies of components \ntailored to accomplish specific high -lev
el tasks. These pre -\ndefined chains streamline the initial setup proce
ss and serve as \nan ideal starting point for your projects. The MindGu
ide Bot \nuses below components from LangChain . \nA. ChatModel \nWithi
n LangChain, a ChatModel is a specific kind of \nlanguage model crafted
to manage conversational \ninteractions. Unlike traditional language mod
els that take one \nstring as input and generate a single string as outp
ut, \nChatModels operate with a list of mes sages as input, \ngenerating
a message as output. \nEach message in the list has two parts: the cont
ent and the \nrole. The content is the actual text or substance of the m
essage, \nwhile the role denotes the role or source of the message (such
\nas "User," "Assistant," "System," etc.). \nThis approach with ChatMod
els opens the door to more \ndynamic and interactive conversations with
the language \nmodel. It empowers the creation of chatbot applications,
\ncustomer support systems, or any other application involving \nmulti -
turn conversations. We utilized the ChatOpenAI \nChatModel to create Min
dGuide chatbots specifically \ndesigned to function as mental health the
rapists. In our \ninteraction with OpenAI, we opted for an OpenAI API ke
y to \nengage with the ChatGpt3 turbo model and utilized a \ntemperature
value of 0.5. The steps to create an OpenAI API \nkey are outlined [9].
B. Message \nIn the context of LangChain, messages [10] refer to a lis
t of \nmessages that are used as input when interacting with a \nChatMod
el. Each message in the list represents a specific turn \nor exchange in
a conversation. Each message in the messages \nlist typically consists
of two components: \n• content: This represents the actual text or cont
ent of \nthe message. It can be a user query, a system \ninstruction, or
any other relevant information. \n• role: This represents the role or s
ource of the \nmessage. It defines who is speaking or generating \nthe m
essage. Common roles include "User", \n"Assistant", "System", or any oth
er custom role you \ndefine . \nThe chat model interface is based around
messages rather \nthan raw text. The types of messages supported in Lang
Chain \nare Systen Message, HumanMessage, and AIMessage . \nSystemMessag
e is the ChatMessage coming from the system \nin its LangChain templat
e as illustrated in Figure 1. Human \nMessage is a ChatMessage coming
from a human/user. \nAIMessage is a ChatMessage coming from an AI/assi

```
stant as \nillustrated in Figure 2 .  \n \n                    Figure 1.
A System Message illustration   You are a compassionate and experienced
mental \nhealth therapist with a proven track record of \nhelping patien
ts overcome anxiety and other mental \nhealth challenges. Your primary o
bjective is to \nsupport the patient in addressing their concerns \nand
guiding th em towards positive change. In this \ninteractive therapy ses
sion, you will engage with \nthe patient by asking open -ended questions
, \nactively listening to their responses, and providing \nempathetic fe
edback. Your approach is \ncollaborative, and you strive to cr eate a sa
fe and \nnon-judgmental space for the patient to share their \nthoughts
and feelings.  \nAs the patient shares their struggles, you will \nprovi
de insightful guidance and evidence -based \nstrategies tailored to thei
r unique needs. You may \nalso offer practical exercises or resources to
help \nthem manage their symptoms and improve their \nmental wellbe ing.
When necessary, you will gently \nredirect the conversation back to the
patient\'s \nprimary concerns related to anxiety, mental health, \nor fa
mily issues. This ensures that each session is \nproductive and focused
on addressing the most \npressing issues. Thro ughout the session, you
\nremain mindful of the patient\'s emotional state and \nadjust your app
roach accordingly.  \nYou recognize that everyone\'s journey is \ndiffer
ent, and that progress can be incremental.  \nBy building trust and fost
ering a strong \ntherapeutic relationship, you empower the patient \nto
take ownership of their growth and development. \nAt the end of the sess
ion, you will summarize key \npoints from your discussion, highlighting
the \npatient\'s strength s and areas for improvement.  \nTogether, you
will set achievable goals for future \nsessions, reinforcing a sense of
hope and \nmotivation. Your ultimate goal is to equip the \npatient with
the tools and skills needed to navigate \nlife\'s challenges with confid
ence and resilience . ')
```

Split the document and store the embeddings into a vector database.

In [39]:
```
# Split
chunks_pdf = text_splitter(pdf_data, 500, 20)

# VectorDB
ids = vectordb.get()["ids"]
vectordb.delete(ids) # We need to delete existing embeddings from previou
vectordb = Chroma.from_documents(documents=chunks_pdf, embedding=watsonx_
```

The `MultiQueryRetriever` function from LangChain is used.

In [40]:
```
from langchain.retrievers.multi_query import MultiQueryRetriever

query = "What does the paper say about langchain?"

retriever = MultiQueryRetriever.from_llm(
    retriever=vectordb.as_retriever(), llm=llm()
)
```

Set logging for the queries.

In [41]:
```
import logging

logging.basicConfig()
logging.getLogger("langchain.retrievers.multi_query").setLevel(logging.IN
```

In [43]:
```
docs = retriever.invoke(query)
```

```
docs
```

```
INFO:langchain.retrievers.multi_query:Generated queries: ['1. What informa
tion does the paper provide regarding langchain?', '', '    2. What insigh
ts about langchain are discussed in the paper?', '', '    3. What details
about langchain are covered in the paper?']
```

Out[43]:
```
[Document(metadata={'page': 5, 'source': 'https://cf-courses-data.s3.u
s.cloud-object-storage.appdomain.cloud/ioch1wsxkfqgfLLgmd-6Rw/langchain-
paper.pdf'}, page_content='[11] LangChain's Large Language Model Chain,
\nhttps://python.langchain.com/docs/modules/chains/foundational/llm_c\nh
ain (accessed Nov. 29, 2023).  \n[12] Streamlit, https://streamlit.io/
(accessed Nov. 29, 2023).'),
 Document(metadata={'page': 1, 'source': 'https://cf-courses-data.s3.u
s.cloud-object-storage.appdomain.cloud/ioch1wsxkfqgfLLgmd-6Rw/langchain-
paper.pdf'}, page_content='• content: This represents the actual text or
content of \nthe message. It can be a user query, a system \ninstructio
n, or any other relevant information.  \n• role: This represents the rol
e or source of the \nmessage. It defines who is speaking or generating
\nthe message. Common roles include "User", \n"Assistant", "System", or
any other custom role you \ndefine . \nThe chat model interface is based
around messages rather \nthan raw text. The types of messages supported
in LangChain'),
 Document(metadata={'page': 0, 'source': 'https://cf-courses-data.s3.u
s.cloud-object-storage.appdomain.cloud/ioch1wsxkfqgfLLgmd-6Rw/langchain-
paper.pdf'}, page_content='the use of contextualized pretrained language
models  [4], \nwhich have garnered substantial attention for their \neff
ectiveness in various text processing tasks.  \nThis paper delves into t
he application of these recent \nadvancements in pretrained contextualiz
ed large language \nmodels to introduce MindGuide , an innovative chatbo
t \ndesigned to function as a mental health assistant for \nindividuals
in need of guidance and support in these critical'),
 Document(metadata={'page': 0, 'source': 'https://cf-courses-data.s3.u
s.cloud-object-storage.appdomain.cloud/ioch1wsxkfqgfLLgmd-6Rw/langchain-
paper.pdf'}, page_content='* corresponding author - jkim72@kent.edu  Rev
olutionizing Mental Health Care through \nLangChain: A Journey with a La
rge Language \nModel\nAditi Singh  \n Computer Science   \n Cleveland St
ate University   \n a.singh22@csuohio.edu  Abul Ehtesham   \nThe Davey T
ree Expert \nCompany   \nabul.ehtesham@davey.com  Saifuddin Mahmud  \nCo
mputer Science & \nInformation Systems   \n Bradley University  \nsmahmu
d@bradley.edu   Jong -Hoon Kim * \n Computer Science,  \nKent State Univ
ersity,  \njkim72@kent.edu'),
 Document(metadata={'page': 1, 'source': 'https://cf-courses-data.s3.u
s.cloud-object-storage.appdomain.cloud/ioch1wsxkfqgfLLgmd-6Rw/langchain-
paper.pdf'}, page_content='This approach with ChatModels opens the door
to more \ndynamic and interactive conversations with the language \nmode
l. It empowers the creation of chatbot applications, \ncustomer support
systems, or any other application involving \nmulti -turn conversations.
We utilized the ChatOpenAI \nChatModel to create MindGuide chatbots spec
ifically \ndesigned to function as mental health therapists. In our \nin
teraction with OpenAI, we opted for an OpenAI API key to'),
 Document(metadata={'page': 0, 'source': 'https://cf-courses-data.s3.u
s.cloud-object-storage.appdomain.cloud/ioch1wsxkfqgfLLgmd-6Rw/langchain-
paper.pdf'}, page_content='II. LANGCHAIN  \nLangChain , with its open -s
ource essence, emerges as a \npromising solution, aiming to simplify the
complex process of \ndeveloping applications powered by large language m
odels \n(LLMs) . This framework though the rapid delivery of building \n
blocks and pre -built chains for building large language model \napplica
tions shows the easy way developers can do it .')]
```

From the log results, you can see that the LLM generated three additional queries from

different perspectives based on the given query.

The returned results are the union of the results from each query.

Self-Querying Retriever

A Self-Querying Retriever, as the name suggests, has the ability to query itself. Specifically, given a natural language query, the retriever uses a query-constructing LLM chain to generate a structured query. It then applies this structured query to its underlying vector store. This enables the retriever to not only use the user-input query for semantic similarity comparison with the contents of stored documents but also to extract and apply filters based on the metadata of those documents.

The following code demonstrates how to use a Self-Querying Retriever.

In [44]:
```
from langchain_core.documents import Document
from langchain.chains.query_constructor.base import AttributeInfo
from langchain.retrievers.self_query.base import SelfQueryRetriever
from lark import lark
```

A couple of document pieces have been prepared where the `page_content` contains descriptions of movies, and the `meta_data` includes different attributes for each movie, such as `year`, `rating`, `genre`, and `director`. These attributes are crucial in the Self-Querying Retriever, as the LLM will use the metadata information to apply filters during the retrieval process.

In [46]:
```
docs = [
    Document(
        page_content="A bunch of scientists bring back dinosaurs and mayh
        metadata={"year": 1993, "rating": 7.7, "genre": "science fiction"
    ),
    Document(
        page_content="Leo DiCaprio gets lost in a dream within a dream wi
        metadata={"year": 2010, "director": "Christopher Nolan", "rating"
    ),
    Document(
        page_content="A psychologist / detective gets lost in a series of
        metadata={"year": 2006, "director": "Satoshi Kon", "rating": 8.6}
    ),
    Document(
        page_content="A bunch of normal-sized women are supremely wholeso
        metadata={"year": 2019, "director": "Greta Gerwig", "rating": 8.3
    ),
    Document(
        page_content="Toys come alive and have a blast doing so",
        metadata={"year": 1995, "genre": "animated"},
    ),
    Document(
        page_content="Three men walk into the Zone, three men walk out of
        metadata={
            "year": 1979,
            "director": "Andrei Tarkovsky",
            "genre": "thriller",
            "rating": 9.9,
```

```
        },
    ),
]
```

Now you can instantiate your retriever. To do this, you'll need to provide some upfront information about the metadata fields that your documents support and a brief description of the document contents.

In [ ]:
```
metadata_field_info = [
    AttributeInfo(
        name="genre",
        description="The genre of the movie. One of ['science fiction', '
        type="string",
    ),
    AttributeInfo(
        name="year",
        description="The year the movie was released",
        type="integer",
    ),
    AttributeInfo(
        name="director",
        description="The name of the movie director",
        type="string",
    ),
    AttributeInfo(
        name="rating", description="A 1-10 rating for the movie", type="f
    ),
]
```

Store the document's embeddings into a vector database.

In [ ]:
```
vectordb = Chroma.from_documents(docs, watsonx_embedding())
```

Use the `SelfQueryRetriever` .

In [ ]:
```
document_content_description = "Brief summary of a movie."

retriever = SelfQueryRetriever.from_llm(
    llm(),
    vectordb,
    document_content_description,
    metadata_field_info,
)
```

Now you can actually try using your retriever.

In [ ]:
```
# This example only specifies a filter
retriever.invoke("I want to watch a movie rated higher than 8.5")
```

In [ ]:
```
# This example specifies a query and a filter
retriever.invoke("Has Greta Gerwig directed any movies about women")
```

When running the following cell, you might encounter some errors or blank content. This is because the LLM cannot get the answer at first. Don't worry; if you re-run it several times, you will get the answer.

In [ ]:
```
# This example specifies a composite filter
retriever.invoke("What's a highly rated (above 8.5) science fiction film?
```

Parent Document Retriever

When splitting documents for retrieval, there are often conflicting desires:

1. You may want to have small documents so that their embeddings can most accurately reflect their meaning. If the documents are too long, the embeddings can lose meaning.
2. You want to have long enough documents so that the context of each chunk is retained.

The `ParentDocumentRetriever` strikes that balance by splitting and storing small chunks of data. During retrieval, it first fetches the small chunks but then looks up the parent IDs for those chunks and returns those larger documents.

In [ ]:
```
from langchain.retrievers import ParentDocumentRetriever
from langchain_text_splitters import CharacterTextSplitter
from langchain.storage import InMemoryStore
```

In [ ]:
```
# Set two splitters. One is with big chunk size (parent) and one is with
parent_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=20
child_splitter = CharacterTextSplitter(chunk_size=200, chunk_overlap=20,
```

In [ ]:
```
vectordb = Chroma(
    collection_name="split_parents", embedding_function=watsonx_embedding
)
#vectordb = Chroma.from_documents(documents=chunks_pdf, embedding=watsonx
# The storage layer for the parent documents
store = InMemoryStore()
```

In [ ]:
```
retriever = ParentDocumentRetriever(
    vectorstore=vectordb,
    docstore=store,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
```

In [ ]:
```
retriever.add_documents(txt_data)
```

Ignore the warnings as the documents before chunking have variable lengths.

These are the number of large chunks:

In [ ]:
```
len(list(store.yield_keys()))
```

Let's make sure the underlying vector store still retrieves the small chunks.

In [ ]:
```
sub_docs = vectordb.similarity_search("smoking policy")
```

In [ ]:
```
print(sub_docs[0].page_content)
```

Then, retrieve the relevant large chunk.

In [ ]:
```
retrieved_docs = retriever.invoke("smoking policy")
print(retrieved_docs[0].page_content)
```

Exercises

Exercise 1

Retrieve Top 2 Results Using a Vector Store-Backed Retriever

Retrieve the top two results for the company policy document for the query "smoking policy" using the Vector Store-Backed Retriever.

In [ ]:
```
# Your code here
```

▸ Click here for the solution

Exercise 2

Self-Querying Retriever for a Query

Use the Self-Querying Retriever to invoke a query with a filter.

In [ ]:
```
# Your code here
```

▸ Click here for the solution

Authors

Kang Wang is a Data Scientist in IBM. He is also a PhD Candidate in the University of Waterloo.

Fateme Akbari is a Ph.D. candidate in Information Systems at McMaster University and data scientist at IBM with demonstrated research experience in Machine Learning and NLP.

Other Contributors

Joseph Santarcangelo has a Ph.D. in Electrical Engineering, his research focused on using machine learning, signal processing, and computer vision to determine how videos impact human cognition. Joseph has been working for IBM since he completed his PhD.

Ricky Shi is a Data Scientist at IBM, specializing in deep learning, computer vision, and Large Language Models. He applies advanced machine learning and generative AI

techniques to solve complex challenges across various sectors. As an enthusiastic mentor, Ricky is committed to helping colleagues and peers master technical intricacies and drive innovation.

Wojciech "Victor" Fulmyk is a Data Scientist at IBM

Change Log

▸ Click here for the changelog

Copyright © IBM Corporation. All rights reserved.